Leveraging Graph Neural Networks to Uncover Illicit Financial Networks

Final Project Report
Graph Neural Networks Course

Team Members

Shivam Bansal (240976)
Naivedya Jain (240678)
Paveet Singh Saluja (240742)
Shourya Mathur (240991)

Dataset: Elliptic Bitcoin Transaction Dataset
Framework: PyTorch Geometric
Submitted: March 2025

Contents

1 Overall Project Summary **2**

2 Details of the Dataset **2**
2.1 Overview . . . 2
2.2 Feature Space . . . 3
2.3 Class Imbalance . . . 3
2.4 Temporal Dynamics . . . 3

3 Details of Data Preprocessing **3**
3.1 Step 1 — Label Encoding . . . 3
3.2 Step 2 — Node Index Mapping . . . 4
3.3 Step 3 — Feature Normalization . . . 4
3.4 Step 4 — Train / Validation / Test Masking . . . 4
3.5 Step 5 — PyTorch Geometric Data Object . . . 4

4 Feature Extraction and Weighing Methods **5**
4.1 Input Features . . . 5
4.2 Ablation Study — Structural vs. Hand-Crafted Features . . . 5
4.3 Post-Hoc Feature Attribution with GNNExplainer . . . 5

5 List of Libraries Used **6**

6 Architecture of the Models Employed **6**
6.1 Shared Design Principles . . . 6
6.2 Model 1: Graph Convolutional Network (GCN) . . . 7
6.3 Model 2: GraphSAGE . . . 7
6.4 Model 3: Graph Attention Network (GAT) . . . 8
6.5 Focal Loss . . . 8

7 Final Results Achieved **9**
7.1 Test Set Performance . . . 9
7.2 Per-Model Classification Reports . . . 9
7.2.1 GCN . . . 9
7.2.2 GraphSAGE . . . 9
7.2.3 GAT . . . 10
7.2.4 XGBoost (Baseline) . . . 10
7.3 Key Observations . . . 10

8 Conclusions, Improvements and Limitations **10**
8.1 Conclusions . . . 10
8.2 Improvements Over Legacy Approaches . . . 11
8.3 Limitations . . . 11

9 Sources and References **12**

1 Overall Project Summary

Financial crimes—from money laundering to synthetic identity fraud—are rarely isolated events. They are driven by coordinated rings employing sophisticated network topologies such as *smurfing* (breaking large sums into smaller, undetectable transactions) and *peeling chains* (rapidly moving funds through intermediary wallets) to obscure illicit cash flows.

Traditional Anti-Money Laundering (AML) systems rely on rule-based engines or tabular Machine Learning algorithms (Random Forests, XGBoost) that treat each transaction as an independent, identically distributed (i.i.d.) data point. This structural assumption blinds them to the multi-hop relational patterns that define fraud rings.

This project frames financial fraud detection as a **Graph Representation Learning** problem. We model the financial ecosystem as a directed graph $G = (V, E)$, where each node $v \in V$ is a Bitcoin transaction and each edge $e \in E$ represents a payment flow. We apply three Graph Neural Network (GNN) architectures on the **Elliptic Dataset**—a real-world, large-scale cryptocurrency transaction graph—and benchmark them against an XGBoost tabular baseline.

The key contributions of this project are:

- Graph construction from raw tabular transaction data into a PyTorch Geometric `Data` object.
- Implementation and comparison of GCN, GraphSAGE, and GAT architectures with Focal Loss to handle severe class imbalance.
- Ablation study demonstrating GNN's ability to learn structural features automatically from graph topology.
- Post-hoc explainability via GNNExplainer to produce interpretable fraud evidence for compliance.

2 Details of the Dataset

2.1 Overview

The **Elliptic Dataset** is the benchmark for graph-based anomaly detection in financial networks, derived from real Bitcoin blockchain data. It represents a directed, homogeneous transaction flow graph.

Table 1: Elliptic Dataset — Core Statistics

Property	**Value**
Domain	Cryptocurrency (Bitcoin) transaction flows
Graph Type	Directed, homogeneous
Nodes (transactions)	203,769
Edges (payment flows)	234,355
Node Feature Dimensions	166
Time Steps	49 (approx. 2 weeks each)
Licit transactions	42,019 (20.62%)
Illicit transactions	4,545 (2.23%)
Unlabelled transactions	157,205 (77.15%)

2.2 Feature Space

Each node carries **166 features** organized in two groups:

94 Local Features (columns 3–95) Transaction-specific attributes such as timestamp, transaction fee, volume, and number of inputs/outputs. These are directly observable from the blockchain record.

72 Aggregated Structural Features (columns 96–167) Neighborhood statistics computed from one-hop neighbors: minimum/maximum/mean fee, volume, and degree of adjacent transactions. These provide traditional manual network context.

2.3 Class Imbalance

The dataset is severely imbalanced: illicit transactions constitute only **2.23%** of all labeled nodes. Because of this, standard accuracy is an invalid metric—a trivial model predicting "licit" for everything achieves $> 97\%$ accuracy. This project relies exclusively on **Macro F1-Score** and **PR-AUC** as evaluation metrics.

2.4 Temporal Dynamics

The 49 time steps allow evaluation of model robustness against *concept drift*—fraud patterns can evolve over the approximately 2-year window of data, challenging models to maintain detection performance as adversarial strategies shift.

3 Details of Data Preprocessing

The raw Elliptic data is provided as three separate CSV files. Constructing a GNN-compatible graph required the following pipeline:

3.1 Step 1 — Label Encoding

String class labels were converted to integers: `'1'` (illicit) $\rightarrow 1$, `'2'` (licit) $\rightarrow 0$, `'unknown'` $\rightarrow -1$. Nodes with label -1 participate in message passing but are excluded from loss computation and evaluation.

3.2 Step 2 — Node Index Mapping

Raw transaction IDs (large, non-contiguous integers) were re-indexed into a contiguous integer space $[0, 203768]$. This is required by PyTorch Geometric's sparse adjacency representation.

Listing 1: Node ID remapping

```
tx_ids   = df['tx_id'].values
txid_map = {txid: i for i, txid in enumerate(tx_ids)}
# Edge list mapped through the same dictionary
src = raw_edges['txId1'].map(txid_map).astype(int).values
dst = raw_edges['txId2'].map(txid_map).astype(int).values
edge_index = torch.tensor([src, dst], dtype=torch.long)
```

3.3 Step 3 — Feature Normalization

All 166 features were normalized using `sklearn.preprocessing.StandardScaler` (zero mean, unit variance). Normalization prevents large-magnitude features from dominating gradient updates and accelerates convergence.

Listing 2: Standard scaling of node features

```
scaler = StandardScaler()
X_std  = scaler.fit_transform(X_raw)  # shape: (203769, 166)
```

3.4 Step 4 — Train / Validation / Test Masking

In graph learning the full graph structure is preserved during training; supervision is applied via boolean masks. Only the 46,564 labeled nodes were split using a stratified 70/15/15 partition to maintain the natural class ratio across all sets.

Table 2: Partition Sizes

Partition	Nodes	Fraction
Training	32,594	70%
Validation	6,985	15%
Test	6,985	15%

3.5 Step 5 — PyTorch Geometric Data Object

All tensors were assembled into a single `Data` object, which was moved to GPU (`cuda`) for training:

Listing 3: Graph object construction

```
graph = Data(
    x          = torch.tensor(X_std, dtype=torch.float),  # (203769, 166)
    edge_index = edge_index,                              # (2, 234355)
```

```
    y           = y_tensor,
    train_mask  = tr_mask,
    val_mask    = val_mask,
    test_mask   = te_mask,
).to(DEVICE)
```

4 Feature Extraction and Weighing Methods

4.1 Input Features

No additional manual feature engineering was applied beyond what the dataset provides. The 166-dimensional node feature vector (after normalization) is passed directly as input to each GNN.

4.2 Ablation Study — Structural vs. Hand-Crafted Features

A key experiment was run to determine whether the GNN can autonomously learn the structural context encoded in the 72 aggregated features. GraphSAGE was trained twice: once with all 166 features, and once with only the 94 local features.

Table 3: Ablation Study: Full vs. Local Features (GraphSAGE)

Configuration	**Macro F1**	**PR-AUC**
GraphSAGE (94 local features only)	0.9029	0.8875
GraphSAGE (full 166 features)	0.9238	0.9308
Δ Gap	0.0209	0.0433

The small but consistent gap (ΔF1 $\approx$ 0.02) indicates that while the hand-crafted aggregated features do add some complementary signal, the GNN independently recovers much of the structural information purely through message passing over the graph topology. This validates the core premise of the GNN approach.

4.3 Post-Hoc Feature Attribution with GNNExplainer

For explainability, GNNExplainer was applied to the best-performing GraphSAGE model. For a representative illicit node (index 36965), the top 10 most influential features were identified:

Table 4: Top 10 Features by GNNExplainer Attribution Score

Rank	Feature	Importance Score
1	feat_113	0.7474
2	feat_149	0.7424
3	feat_53	0.7322
4	feat_55	0.7318
5	feat_90	0.7278
6	feat_52	0.7204
7	feat_89	0.7063
8	feat_137	0.6296
9	feat_155	0.6230
10	feat_154	0.5568

GNNExplainer outputs a node-level mask and an edge mask, identifying which subgraph and features most heavily influenced the fraud prediction. This provides human-interpretable evidence for compliance analysts.

5 List of Libraries Used

Table 5: Python Libraries and Their Roles

Library	Version	Purpose
`PyTorch`	2.10.0+cu128	Deep learning framework; tensor operations and autograd
`torch-geometric`	Latest	GNN layer implementations (GCNConv, SAGEConv, GATConv, GNNExplainer)
`torch-scatter`	2.1.2	Sparse scatter operations required by PyG
`torch-sparse`	Latest	Sparse matrix operations required by PyG
`NumPy`	Latest	Numerical array operations
`pandas`	Latest	Data ingestion and manipulation of CSV files
`scikit-learn`	Latest	StandardScaler, train_test_split, evaluation metrics
`XGBoost`	Latest	Tabular baseline classifier
`matplotlib`	Latest	Plotting learning curves and bar charts
`seaborn`	Latest	Confusion matrix heatmaps and styled bar plots

All models were trained on GPU (`cuda`, PyTorch 2.10.0+cu128).

6 Architecture of the Models Employed

6.1 Shared Design Principles

All three GNN models share the following design:

- **3 graph convolutional layers** with hidden dimension 128.

- **Dropout** ($p = 0.45$) applied after the first two layers to regularize against overfitting.
- **Output dimension** of 2 (binary: licit vs. illicit).
- **Focal Loss** as the training objective (see Section 6.5).
- **Adam optimizer**, learning rate 5×10^{-3}, L2 weight decay 5×10^{-4}.
- **StepLR** scheduler halving the learning rate every 50 epochs.
- **200 training epochs** with best-validation-F1 checkpointing.

6.2 Model 1: Graph Convolutional Network (GCN)

GCN applies a symmetric, normalized Laplacian-based spectral convolution. At each layer ℓ, the node representation $h_v^{(\ell)}$ is updated as:

$$H^{(\ell+1)} = \sigma\Big(\tilde{D}^{-1/2}\tilde{A}\,\tilde{D}^{-1/2}\,H^{(\ell)}\,W^{(\ell)}\Big) \tag{1}$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops, $\tilde{D}$ is the corresponding degree matrix, and $W^{(\ell)}$ is a learnable weight matrix.

Listing 4: GCN Architecture

```
class GCNModel(nn.Module):
    def __init__(self, n_input, n_hidden, n_classes, drop_rate
       =0.45):
        self.conv_in  = GCNConv(n_input,  n_hidden)
        self.conv_mid = GCNConv(n_hidden, n_hidden)
        self.conv_out = GCNConv(n_hidden, n_classes)
```

Rationale: GCN serves as the spectral baseline, establishing the performance floor for pure neighborhood averaging.

6.3 Model 2: GraphSAGE

GraphSAGE (Hamilton et al., 2017) performs *inductive* learning via neighborhood sampling and concatenation-based aggregation:

$$h_v^{(\ell+1)} = \sigma\big(W^{(\ell)} \cdot \big[h_v^{(\ell)} \,\|\, \mathrm{MEAN}\big(\{h_u^{(\ell)} : u \in \mathcal{N}(v)\}\big)\big]\big) \tag{2}$$

The concatenation of the node's own embedding with the aggregated neighbor mean allows the model to retain individual node identity while absorbing neighborhood context.

Listing 5: GraphSAGE Architecture

```
class SAGEModel(nn.Module):
    def __init__(self, n_input, n_hidden, n_classes, drop_rate
       =0.45):
        self.sage_in  = SAGEConv(n_input,  n_hidden)
        self.sage_mid = SAGEConv(n_hidden, n_hidden)
        self.sage_out = SAGEConv(n_hidden, n_classes)
```

Rationale: GraphSAGE is particularly suited to large-scale graphs and real-time inference because it can generalize to unseen nodes without retraining (inductive capability), a critical requirement for production transaction screening.

6.4 Model 3: Graph Attention Network (GAT)

GAT (Veličković et al., 2018) replaces fixed averaging with learned, dynamic attention weights α_{vu}:

$$h_v^{(\ell+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu}\, W\, h_u^{(\ell)}\right) \tag{3}$$

$$\alpha_{vu} = \frac{\exp\big(\mathrm{LeakyReLU}(a^T[Wh_v \,\|\, Wh_u])\big)}{\sum_{k \in \mathcal{N}(v)} \exp(\mathrm{LeakyReLU}(a^T[Wh_v \,\|\, Wh_k]))} \tag{4}$$

The implementation uses 4 attention heads with feature concatenation in the first two layers, and a single head with feature averaging in the output layer.

Listing 6: GAT Architecture

```
class GATModel(nn.Module):
    def __init__(self, n_input, n_hidden, n_classes, num_heads=4,
        drop_rate=0.45):
        self.gat1 = GATConv(n_input,            n_hidden, heads
           =4, concat=True)
        self.gat2 = GATConv(n_hidden * 4,       n_hidden, heads
           =4, concat=True)
        self.gat3 = GATConv(n_hidden * 4, n_classes, heads=1,
           concat=False)
```

Rationale: GAT's attention mechanism allows the model to dynamically down-weight benign neighbors and focus on suspicious connections, which is theoretically well-motivated for fraud ring detection.

6.5 Focal Loss

Standard cross-entropy loss is dominated by the majority class. Focal Loss (Lin et al., 2017) down-weights well-classified examples and forces the model to focus on hard, misclassified cases (i.e., the illicit minority):

$$\mathcal{L}_{\text{focal}} = -\alpha\,(1 - p_t)^{\gamma}\,\log(p_t) \tag{5}$$

where p_t is the probability assigned to the ground-truth class, $\alpha = 0.25$ is the class weighting factor, and $\gamma = 2.0$ is the focusing parameter. When a sample is misclassified, $p_t \to 0$ and the focal weight $(1 - p_t)^{\gamma} \to 1$, applying full loss. When correctly classified with high confidence, $p_t \to 1$ and the loss is suppressed.

7 Final Results Achieved

7.1 Test Set Performance

Table 6: Test Set Results — All Models (ranked by Macro F1)

Model	Macro F1	PR-AUC	Illicit Precision	Illicit Recall
XGBoost (Tabular Baseline)	**0.9770**	**0.9858**	0.99	0.93
GraphSAGE	0.9238	0.9308	0.95	0.78
GAT	0.8227	0.6898	0.74	0.63
GCN	0.8180	0.7712	0.83	0.56

7.2 Per-Model Classification Reports

7.2.1 GCN

Table 7: GCN — Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
Licit	0.95	0.99	0.97	6306
Illicit	0.83	0.56	0.67	679
Macro Avg	0.89	0.77	0.82	6985
Accuracy		0.95		

7.2.2 GraphSAGE

Table 8: GraphSAGE — Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
Licit	0.98	1.00	0.99	6306
Illicit	0.95	0.78	0.86	679
Macro Avg	0.97	0.89	0.92	6985
Accuracy		0.98		

7.2.3 GAT

Table 9: GAT — Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
Licit	0.96	0.98	0.97	6306
Illicit	0.74	0.63	0.68	679
Macro Avg	0.85	0.80	0.82	6985
Accuracy		0.94		

7.2.4 XGBoost (Baseline)

Table 10: XGBoost — Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
Licit	0.99	1.00	1.00	6306
Illicit	0.99	0.93	0.96	679
Macro Avg	0.99	0.96	0.98	6985
Accuracy		0.99		

7.3 Key Observations

- **GraphSAGE was the best-performing GNN**, achieving Macro F1 = 0.9238 and PR-AUC = 0.9308, substantially outperforming both GCN (0.818) and GAT (0.823).
- **XGBoost outperformed all GNN models** (F1 = 0.977, PR-AUC = 0.986), demonstrating that on this specific dataset—which already includes 72 hand-crafted structural features—tabular methods remain highly competitive.
- **GCN's recall of 0.56** on illicit transactions is problematically low in a business context: nearly half of all fraud goes undetected.
- **GAT underperformed expectations**, possibly due to attention over-smoothing on large, sparse graphs or the full-batch training regime limiting its optimization dynamics.
- The **ablation study confirms** that GraphSAGE with only local features still achieves F1 = 0.903, demonstrating meaningful structural learning from graph topology alone.

8 Conclusions, Improvements and Limitations

8.1 Conclusions

This project successfully demonstrated the feasibility of Graph Neural Networks for illicit transaction detection on the Elliptic Dataset. Among the three architectures, **GraphSAGE emerged as the optimal GNN model**, offering the best balance of Macro F1, PR-AUC, and scalability. Its inductive learning capability—the ability to

generate embeddings for unseen nodes without retraining the full graph—makes it the most viable candidate for real-time production deployment.

The XGBoost baseline's superior numerical performance is partly attributed to the Elliptic Dataset's inclusion of 72 pre-computed aggregated structural features. In a realistic deployment scenario where these would need to be recomputed in real-time or are unavailable, the GNN's advantage would be more pronounced.

8.2 Improvements Over Legacy Approaches

Beyond I.I.D. GNNs natively exploit the graph structure, capturing multi-hop relationships that tabular models require expensive hand-engineering to approximate.

Automatic Feature Learning The ablation study showed the GNN recovers most structural context (F1 ≈ 0.90) using only raw transaction features, eliminating the manual feature engineering bottleneck.

Scalable Inductive Inference GraphSAGE's neighborhood sampling approach avoids full-batch gradient computation, enabling scalability to production-scale graphs with millions of nodes.

Interpretability GNNExplainer provides subgraph-level and feature-level explanations for each prediction, directly addressing the "black box" compliance concern in financial regulation.

Minority Class Focus Focal Loss significantly improved illicit recall compared to using standard cross-entropy, reducing the false negative rate which is the highest-cost error type in AML.

8.3 Limitations

Full-Batch Training All three GNN models were trained in full-batch mode, loading the entire 203K-node graph into GPU memory. This does not scale to production graphs with millions of transactions; mini-batch neighborhood sampling (GraphSAGE's original formulation) is required.

Static Graph Assumption The GNN model is retrained on a static snapshot. Production systems require incremental or streaming graph updates as new transactions arrive.

Unlabelled Nodes 77% of nodes are unlabelled and excluded from gradient updates. Label propagation or self-supervised pretraining could leverage these nodes to improve embeddings.

Temporal Exploitation The 49 time steps provide a natural temporal signal that was not explicitly exploited. Temporal GNNs (e.g., TGAT, TGN) could model concept drift in fraud patterns more effectively.

XGBoost Parity On this specific dataset, XGBoost's numerical advantage suggests the 72 pre-aggregated structural features are highly informative. Stripping these features before the tabular baseline comparison would provide a fairer head-to-head assessment of structural learning.

Hyperparameter Sensitivity GAT's underperformance warrants deeper hyperparameter exploration (heads, hidden dim, learning rate schedules). The current configuration was shared across all architectures for fair comparison, but may not be optimal for GAT.

9 Sources and References

Dataset

- **Elliptic Dataset**
  https://www.elliptic.co/resources/elliptic-dataset-cryptocurrency-transactions

Research Papers

- **Kipf, T. N., & Welling, M. (2017)**
  Semi-Supervised Classification with Graph Convolutional Networks
  https://arxiv.org/abs/1609.02907
- **Hamilton, W. L., Ying, R., & Leskovec, J. (2017)**
  Inductive Representation Learning on Large Graphs (GraphSAGE)
  https://arxiv.org/abs/1706.02216
- **Veličković, P., et al. (2018)**
  Graph Attention Networks (GAT)
  https://arxiv.org/abs/1710.10903
- **McAuley, J., et al. / Elliptic (2019)**
  Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks
  https://arxiv.org/abs/1908.02591

Libraries & Documentation

- **PyTorch Documentation**
  https://pytorch.org/docs/stable/index.html
- **PyTorch Geometric Documentation**
  https://pytorch-geometric.readthedocs.io
- **Scikit-learn Documentation**
  https://scikit-learn.org/stable/
- **Pandas Documentation**
  https://pandas.pydata.org/docs/
- **NumPy Documentation**
  https://numpy.org/doc/